



[12] 发 明 专 利 说 明 书

[21] ZL 专利号 00102567.8

[45] 授权公告日 2004 年 1 月 28 日

[11] 授权公告号 CN 1136709C

[22] 申请日 2000.3.21 [21] 申请号 00102567.8

[71] 专利权人 华为技术有限公司

地址 518057 广东省深圳市科技园科发路华为
用户服务中心大厦

[72] 发明人 黄 韬

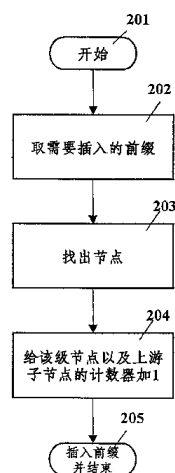
审查员 刘剑波

权利要求书 1 页 说明书 5 页 附图 3 页

[54] 发明名称 IP 查找的一种内存回收方法

[57] 摘要

IP 快速查找方法，即最长前缀匹配方法是路由方法中的一个重要组成部分。本发明在“扩充前缀方法”的基础上，针对路由项删除时，空闲节点内存回收困难的问题，提出“节点计数器”的思想。成功解决了路由修改内存回收问题，达到很好的效果。



1、 一种IP路由查找中的内存回收方法，其特征在于包括以下步骤：

给每个节点数组设一个节点计数器；

当在当前节点数组或当前节点数组的下游节点数组中插入和删除前缀时，节点计数器相应加一或减一；以及

当节点计数器为零时，回收该节点数组以及其下游节点数组所占用的内存。

2、 一如权利要求1所述的方法，其特征在于还包括：

当插入的前缀不能放入现有节点中时，分配内存以用于增加新节点的步骤。

3、 如权利要求2所述的方法，其特征在于：

增加新节点的步骤包括设定增加的节点的大小为 2^n 次的步骤，其中 n 为节点所在级的长度。

4、 如权利要求1，2或3的任一项所述的方法，其特征在于：

给节点设定节点计数器的步骤包括将节点计数器设定在节点数组的开始步骤。

IP查找的一种内存回收方法

技术领域

本发明涉及一种在网络路由器上实现的IP查找方法，尤其涉及IP查找中的内存回收方法。

背景技术

本发明是在“扩充前缀方法”(以下简称CPE方法)的基础上进行的，CPE方法是由Srinivasan提出的一种快速查找方法，论文名称为“Faster IP Lookups using Controlled Prefix Expansion”，出自proc. ACM sigmetrics'98 conf., madison, WI, 第1-11页。方法利用分段压缩的思想减少方法的时间复杂度，使得查找速度大大增加，根据不同的配置，可以在3~32步之间查找完成，下面简单介绍一下CPE方法。

自从1993年采用CIDR（无类域间路由）协议以来，IP路由就包含两个部分<IP前缀，前缀的长度>，前缀的长度范围在0~32之间。对于每一个输入包，IP查找引擎需要查找出匹配的IP前缀集合，并从匹配的前缀集合中查找出最长匹配的前缀。最长匹配的前缀所指示的地址就是查找出来的下一跳的地址。关于CIDR，详细可见RFC1519 Classless Inter-Domain Routing (CEIR): an Address Assignment and Aggregation Strategy. V. Fuller, T. Li, J. Yu, &K. Varadhan. September 1993 以及RFC1817 CIDR and Classful Routing. Y. Rekhter. August 1995。

路由表中的规则可以表示为前缀的形式，前缀的长度在8位和32位之间变化，IP查找方法即找出最长匹配的前缀，由于前缀长度的多样性，是造成查找方法时间复杂度高的一个重要因素。对于前缀最大长度为W，NetBSD (NetBSD 是一种标准的Unix操作系统，该操作系统实现了TCP/IP协议栈，其中包含IP查找的实现)中的二叉树的方法复杂度为 $O(W)$ ，如果前缀最大长度减少为L($L < W$)，则方法复杂度将减少

为 $O(L)$ 。显而易见，减少前缀的最大长度将提高方法的速度。控制前缀扩充方法的基本思想是通过减少前缀长度的多样性来增加方法的步长，从而减少了方法的复杂度，达到快速查找的目的。

任何前缀，可以扩充为指定长度的前缀集合，如对于前缀 10^* ，可以扩充为长度为4的前缀集合 $\{1000^*, 1001^*, 1010^*, 1011^*\}$ ，通过扩充前缀到指定的长度，可以归一化前缀的长度，从而减少了前缀长度的多样性。扩充前缀方法可以把扩充后归一化的前缀长度分为几级，每一级前缀的长度是一样的，通过数组索引就可以找到该级别的节点。

扩充前缀tries树（多叉树）把原来的1比特树的32级结果压缩为3级结构，查找次数缩短为3次，方法的时间复杂度为 $O(3)$ 。从而达到快速查找的目的。假设 $P5 = 0^*$ ， $P1 = 10^*$ ， $P2 = 111^*$ ， $P3 = 11001^*$ ， $P4 = 1^*$ ， $P6 = 1000^*$ ， $P7 = 100000^*$ ， $P8 = 1000000^*$ ，则典型的扩充前缀树的结构如图1所示。

但是在CPE方法中，CPE方法在删除一个路由项后，如果路由项所在的节点数组中没有其他的前缀存在，需要回收节点数组的内存空间，即如果该节点以及节点以下的所有前缀都已经删除，则需要对该节点以及该节点以下的所有节点进行内存回收。如何判断节点中的前缀是否删除完毕是一个问题，CPE方法没有对这个问题进行讨论，通常的做法是进行遍历操作。遍历操作将占用大量的内存操作时间，并且修改频繁时，需要大量的内存，这在高速路由器的场合是不实用的。

发明内容

本发明的目的就是要解决CPE方法中删除路由项的内存回收问题，减少内存操作时间，减少内存的占用。

为了实现以上目的，本发明提出了一种内存回收方法，其特征在于包括以下步骤：给每个节点数组设一个节点计数器；当在当前节点数组或当前节点数组的下游节点数组中插入和删除前缀时，节点计数

器相应加一或减一；以及当节点计数器为零时，回收该节点数组以及其下游节点数组所占用的内存。

本发明特别适用于CPE方法，但是本领域的普通技术人员将能够容易将本发明的原理用到其它的方法中。

另外，本发明的方法还能与其它方法同时使用来进一步改进CPE方法，例如，可以先用LeafPush（叶子下移）方法对CPE方法进行优化，然后再应用本发明。或者本发明的方法可以单独使用。

从以上方法可以看出，当前缀插入到一个节点时，该节点以及该节点的上游节点的节点计数器加一，当前缀从一个节点删除时，该节点以及该节点的上游节点的节点计数器减一。如果一个节点的计数器减为零，则可以删除该节点以及以下的节点。这样就完美的解决了节点删除时的内存回收问题。

附图说明

下面将参照附图对本发明进行更加详细的描述，附图中：

图1是CPE方法中典型的扩充前缀树的结构；

图2是根据本发明采用节点计数器实现的前缀的插入流程图；

图3是根据本发明采用节点计数器实现的前缀的删除流程图。

具体实施方式

下面参照图2来描述根据本发明采用节点计数器实现的前缀的插入流程图。首先对本发明的节点计数器进行描述。在本发明中，节点计数器保存每一个节点数组插入和删除前缀的信息，在本发明中，对应每一个节点数组有一个节点计数器，为P1所对应的节点计数器可以放在节点数组前、后以及其它位置。在放置在其它位置的情况下，需要各计数器与对应的节点数组之间的索引值。在优选实施例中，节点计数器放在节点数组前。在本发明中，节点计数器用来记录插入和删除的前缀项，而不是用来记录在当前节点的插入和删除次数。即只有插入和删除引起前缀项的增加或减少时才引起计数器的增加或减少，如果插入和删除只是引起节点数组的改变而不导致前缀项的增加或减

少，则节点计数器不改变。在本发明中，在当前节点数组中插入和删除前缀时，节点计数器相应加一和减一。在当前节点数组的下游数组插入和删除前缀时，节点计数器相应的加一和减一。在当前节点数组上游插入和删除前缀时，节点计数器不变。当节点计数器为零时，可以删除该节点，即可以回收节点数组的内存。

下面根据图2描述根据本发明一个优选实施例，插入前缀的操作流程图。流程在方框201开始，对系统进行初始化。在图1的情况下，为节点11设定一个计数器C11，为节点21设定计数器C21，为节点22设定计数器C22，为节点31设定计数器C31，根据本发明的计数器的原理，设定各计数器的初始值分别为 $C11 = 8$ （下面有8个前缀P1 - P8）， $C21 = 3$ （有3个前缀P6，P7和P8）， $C22 = 2$ （有两个前缀P2，和P3）以及 $C31 = 2$ （有两个前缀P7和P8）。需要注意的是，这里为了描述的方便描述了初始化，实际中，不需要每次流程开始都需要进行初始化。只需要在系统的开始进行初始化。以后每次流程的开始只需将上次结果作为初始值进行。接着，流程到达方框202。在方框202，取需要插入的前缀，例如，假设要插入的前缀为 $P9 = 1001^*$ 。接着，流程到达方框203，在此判断出该前缀属于节点21，所以流程接着到达方框204，在此，因为节点21是当前节点，其上游节点是节点11，所以给节点计数器C21和C11都加1。之后，流程在方框205插入前缀P9并结束。在方框205可以用任何本领域的普通技术人员所熟悉的现有技术的方法来进行插入，例如这样来进行：先将P9用CPE扩充为 10010^* 和 10011^* ，接着找到10010和10011的位置，并写入P9。

在其它情况下，增加前缀会引起节点的增加，此时，需要分配新的节点并为新的节点增加节点计数器。例如，假如在方框202得到要加入的前缀是 010^* ，则在方框203将判断出前缀不能放入任何已有节点，所以新建例如一个节点23，并为该节点分配一个计数器C23并且为该计数器设定初始值为0。到达方框204后，给计数器C23和C11都加1。最后插入该前缀并且流程结束。在优选实施例中，给一个节点分配的内存长度大小是 2^n ，其中n是节点所在级别的长度。例如在图1的情况

下，若分配一个在第二级的节点，则所分配的内存长度为 $2^3 = 8$ ，因为第二级的长度是 $5 - 2 = 3$ 。同理，分配的第三级的节点大小为 $2^{7-5} = 4$ 。

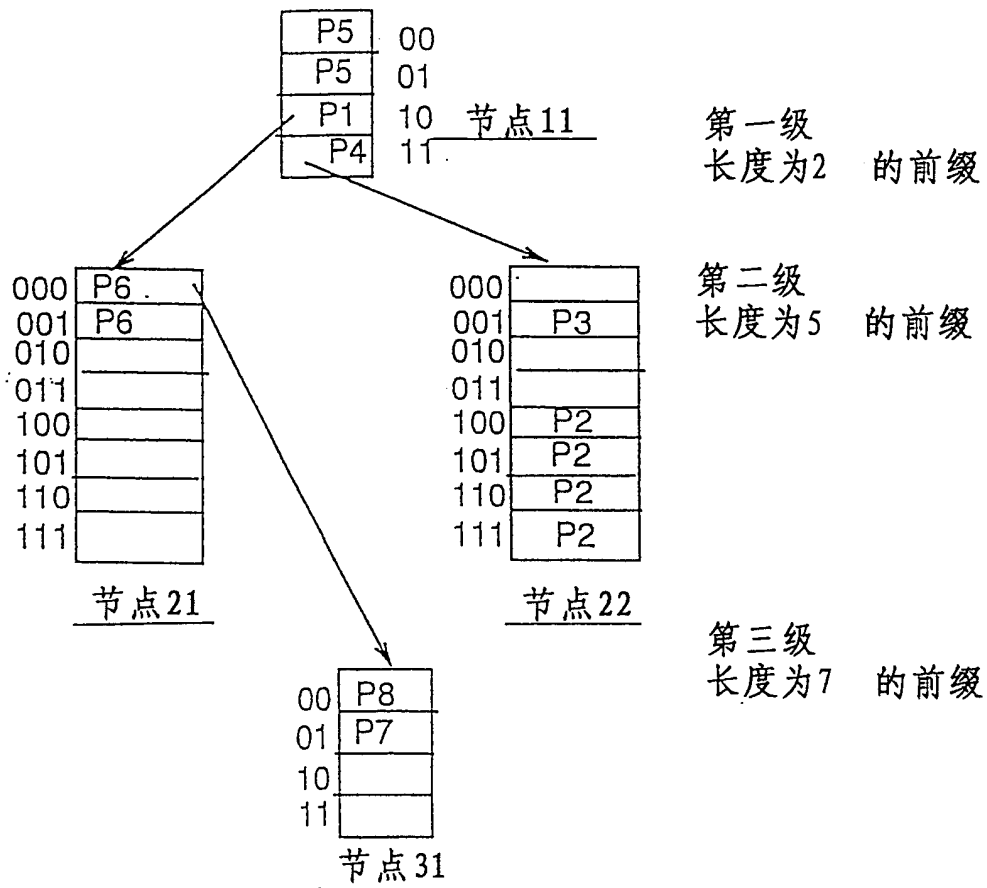
下面参照图3描述根据本发明采用节点计数器实现的前缀的删除流程图。

图3的流程在方框301开始，接着在方框302取需要删除的前缀，例如，需要删除的前缀是p8，接着判断出P8是在第三级的节点C31，并且删除该前缀P8。接着在方框303将该节点的计数器C31减一，在方框304判断出该计数器不为0，接着到达方框306判断是否已到达最后一节点计数器，此时判断的结果为否，所以流程继续到方框303找到第二级的节点21并将该节点的计数器C21减1。接着，该流程又通过方框306和303。在方框303第一级的节点11将该节点的计数器C11减1，在方框304判断出该计数器不为0，到达方框306判断是否已达到最后一节点计数器，判断结果为是，进入方框307，结束该流程退出循环。

在这次循环过程中，没有任何一个节点计数器到达零，所以不用删除任何一个节点。接着假如还要删除节点P7，则在这个过程中节点31的计数器C31变为0，所以将在方框305删除节点31的数组，释放内存。

在图3的实施例中，流程是给每个相关节点计数器减1后就马上接着判断其是否为0，并接着进行相应的处理。在其它实施例中，可以将流程改为先将相关的所有计数器都减1，然后从最低一级的节点计数器开始判断计数器是否为0，若遇到第一个不为0就可以退出循环，因为在低级的节点计数器不为0的情况下，高级的节点计数器不会为0。例如在图1的情况下，在计数器C31不为0的情况下，计数器C21和C11都肯定不会为0，所以可以直接退出循环。

图1



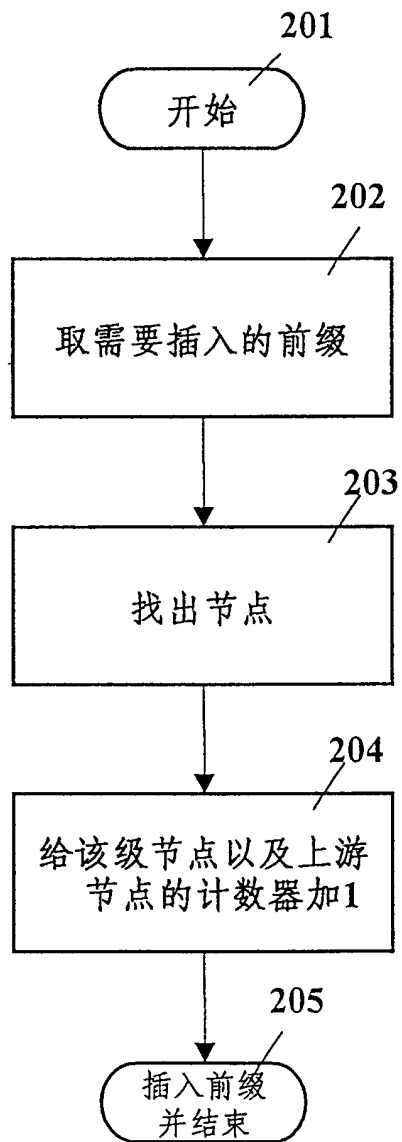


图2

